

EDA And Feature Engineering - Housing Price Prediction

Dataset Information

This dataset contains house sale prices for King County, USA, including Seattle. It includes homes sold between May 2014 and May 2015.

Features:

1. **id** - Unique ID for each house sold
2. **date** - Date of the house sale
3. **price** - Price of each house sold (Target Variable)
4. **bedrooms** - Number of bedrooms
5. **bathrooms** - Number of bathrooms
6. **sqft_living** - Square footage of the apartments interior living space
7. **sqft_lot** - Square footage of the land space
8. **floors** - Number of floors
9. **waterfront** - Whether the apartment was overlooking the waterfront (0/1)
10. **view** - An index from 0 to 4 of how good the view of the property was
11. **condition** - An index from 1 to 5 on the condition of the apartment
12. **grade** - An index from 1 to 13 grading the construction and design of the building
13. **sqft_above** - Square footage of the interior housing space above ground level
14. **sqft_basement** - Square footage of the interior housing space below ground level
15. **yr_built** - Year the house was initially built
16. **yr_renovated** - Year of the house's last renovation
17. **zipcode** - Zipcode area
18. **lat** - Latitude coordinate
19. **long** - Longitude coordinate
20. **sqft_living15** - Square footage of interior living space for the nearest 15 neighbors
21. **sqft_lot15** - Square footage of the land lots of the nearest 15 neighbors

```
from google.colab import files
uploaded = files.upload()
```

[Choose Files](#) Housing - Housing.csv

Housing - Housing.csv(text/csv) - 2359555 bytes, last modified: 4/29/2026 - 100% done
Saving Housing - Housing.csv to Housing - Housing (1).csv

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline

plt.rcParams['figure.figsize'] = (6, 4)
plt.rcParams['figure.dpi'] = 100
```

```
import io
df = pd.read_csv('Housing - Housing.csv')
df.head()
```

	id	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	...	grade	s
0	7229300521	20141013T000000	231300.0	2	1.00	1180	5650	1.0	0	0	...	7	
1	6414100192	20141209T000000	538000.0	3	2.25	2570	7242	2.0	0	0	...	7	
2	5631500400	20150225T000000	180000.0	2	1.00	770	10000	1.0	0	0	...	6	
3	2487200875	20141209T000000	604000.0	4	3.00	1960	5000	1.0	0	0	...	7	
4	1954400510	20150218T000000	510000.0	3	2.00	1680	8080	1.0	0	0	...	8	

5 rows × 21 columns

```
df.tail()
```

	id	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	...	grade
21608	263000018	20140521T000000	360000.0	3	2.50	1530	1131	3.0	0	0	...	
21609	6600060120	20150223T000000	400000.0	4	2.50	2310	5813	2.0	0	0	...	
21610	1523300141	20140623T000000	402101.0	2	0.75	1020	1350	2.0	0	0	...	
21611	291310100	20150116T000000	400000.0	3	2.50	1600	2388	2.0	0	0	...	
21612	1523300157	20141015T000000	325000.0	2	0.75	1020	1076	2.0	0	0	...	

5 rows × 21 columns

Basic Dataset Info

```
## Get the basic info about the data
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21613 entries, 0 to 21612
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   id                    21613 non-null  int64
1   date                 21613 non-null  object
2   price               21613 non-null  float64
3   bedrooms            21613 non-null  int64
4   bathrooms           21613 non-null  float64
5   sqft_living         21613 non-null  int64
6   sqft_lot            21613 non-null  int64
7   floors              21613 non-null  float64
8   waterfront          21613 non-null  int64
9   view                21613 non-null  int64
10  condition            21613 non-null  int64
11  grade                21613 non-null  int64
12  sqft_above           21613 non-null  int64
13  sqft_basement        21613 non-null  int64
14  yr_built             21613 non-null  int64
15  yr_renovated         21613 non-null  int64
16  zipcode              21613 non-null  int64
17  lat                  21613 non-null  float64
18  long                 21613 non-null  float64
19  sqft_living15        21613 non-null  int64
20  sqft_lot15           21613 non-null  int64
dtypes: float64(5), int64(15), object(1)
memory usage: 3.5+ MB
```

```
## Descriptive summary of the dataset
df.describe()
```

	id	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view
count	2.161300e+04	2.161300e+04	21613.000000	21613.000000	21613.000000	2.161300e+04	21613.000000	21613.000000	21613.000000
mean	4.580306e+09	5.401826e+05	3.370795	2.114757	2079.899736	1.510697e+04	1.494309	0.007542	0.234
std	2.876570e+09	3.673619e+05	0.930105	0.770163	918.440897	4.142051e+04	0.539989	0.086517	0.766
min	1.000102e+06	7.500000e+04	0.000000	0.000000	290.000000	5.200000e+02	1.000000	0.000000	0.000
25%	2.123049e+09	3.219500e+05	3.000000	1.750000	1427.000000	5.040000e+03	1.000000	0.000000	0.000
50%	3.904930e+09	4.500000e+05	3.000000	2.250000	1910.000000	7.618000e+03	1.500000	0.000000	0.000
75%	7.308900e+09	6.450000e+05	4.000000	2.500000	2550.000000	1.068800e+04	2.000000	0.000000	0.000
max	9.900000e+09	7.700000e+06	33.000000	8.000000	13540.000000	1.651359e+06	3.500000	1.000000	4.000

```
df.shape
```

(21613, 21)

```
## List all column names
df.columns
```

```
Index(['id', 'date', 'price', 'bedrooms', 'bathrooms', 'sqft_living',
       'sqft_lot', 'floors', 'waterfront', 'view', 'condition', 'grade',
       'sqft_above', 'sqft_basement', 'yr_built', 'yr_renovated', 'zipcode',
       'lat', 'long', 'sqft_living15', 'sqft_lot15'],
      dtype='object')
```

Missing Values & Duplicates

```
## Check missing values in the dataset
df.isnull().sum()
```

	0
id	0
date	0
price	0
bedrooms	0
bathrooms	0
sqft_living	0
sqft_lot	0
floors	0
waterfront	0
view	0
condition	0
grade	0
sqft_above	0
sqft_basement	0
yr_built	0
yr_renovated	0
zipcode	0
lat	0
long	0
sqft_living15	0
sqft_lot15	0

dtype: int64

```
## Check duplicate records
df[df.duplicated()]
```

id	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	...	grade	sqft_above	sqft_basemen
0 rows × 14 columns													

```
## Remove duplicates if any
df.drop_duplicates(inplace=True)
print('Shape after removing duplicates:', df.shape)
```

Shape after removing duplicates: (21613, 21)

Feature Engineering

1. Extract Date Features from `date` column

The `date` column is in format `20141013T000000`. We will extract **Year**, **Month**, and **Day** from it.

```
## Extract Year, Month, Day from date column
df['year'] = df['date'].str[:4].astype(int)
df['month'] = df['date'].str[4:6].astype(int)
df['day'] = df['date'].str[6:8].astype(int)
df.head()
```

	id	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	...	yr_built
0	7229300521	20141013T000000	231300.0	2	1.00	1180	5650	1.0	0	0	...	1955
1	6414100192	20141209T000000	538000.0	3	2.25	2570	7242	2.0	0	0	...	1951
2	5631500400	20150225T000000	180000.0	2	1.00	770	10000	1.0	0	0	...	1933
3	2487200875	20141209T000000	604000.0	4	3.00	1960	5000	1.0	0	0	...	1965
4	1954400510	20150218T000000	510000.0	3	2.00	1680	8080	1.0	0	0	...	1987

5 rows × 24 columns

```
## Drop the original date column
df.drop('date', axis=1, inplace=True)
df.head()
```

	id	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	condition	...	yr_built	yr_renovated
0	7229300521	231300.0	2	1.00	1180	5650	1.0	0	0	3	...	1955	
1	6414100192	538000.0	3	2.25	2570	7242	2.0	0	0	3	...	1951	
2	5631500400	180000.0	2	1.00	770	10000	1.0	0	0	3	...	1933	
3	2487200875	604000.0	4	3.00	1960	5000	1.0	0	0	5	...	1965	
4	1954400510	510000.0	3	2.00	1680	8080	1.0	0	0	3	...	1987	

5 rows × 23 columns

2. Feature Engineering on `yr_renovated`

If `yr_renovated` is 0, it means the house was never renovated. We create a binary feature `was_renovated`.

```
## Create binary feature: was_renovated
df['was_renovated'] = df['yr_renovated'].apply(lambda x: 0 if x == 0 else 1)
df['was_renovated'].value_counts()
```

	count
was_renovated	
0	20699
1	914

dtype: int64

3. Feature Engineering on `sqft_basement`

If `sqft_basement` is 0, it means there is no basement. Create a binary feature `has_basement`.

```
## Create binary feature: has_basement
df['has_basement'] = df['sqft_basement'].apply(lambda x: 0 if x == 0 else 1)
df['has_basement'].value_counts()
```

	count
has_basement	
0	13126
1	8487

dtype: int64

4. House Age Feature

Calculate how old the house is using `yr_built`.

```
## Create house_age feature
df['house_age'] = df['year'] - df['yr_built']
df['house_age'].describe()
```

```

house_age
count    21613.000000
mean      43.317818
std       29.375493
min       -1.000000
25%       18.000000
50%       40.000000
75%       63.000000
max       115.000000

```

dtype: float64

df.info()

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21613 entries, 0 to 21612
Data columns (total 26 columns):
 #   Column              Non-Null Count  Dtype  
---  -
 0   id                  21613 non-null  int64   
 1   price               21613 non-null  float64  
 2   bedrooms            21613 non-null  int64   
 3   bathrooms           21613 non-null  float64  
 4   sqft_living         21613 non-null  int64   
 5   sqft_lot            21613 non-null  int64   
 6   floors              21613 non-null  float64  
 7   waterfront          21613 non-null  int64   
 8   view                21613 non-null  int64   
 9   condition           21613 non-null  int64   
10  grade               21613 non-null  int64   
11  sqft_above          21613 non-null  int64   
12  sqft_basement       21613 non-null  int64   
13  yr_built            21613 non-null  int64   
14  yr_renovated        21613 non-null  int64   
15  zipcode             21613 non-null  int64   
16  lat                 21613 non-null  float64  
17  long                21613 non-null  float64  
18  sqft_living15       21613 non-null  int64   
19  sqft_lot15          21613 non-null  int64   
20  year                21613 non-null  int64   
21  month               21613 non-null  int64   
22  day                 21613 non-null  int64   
23  was_renovated       21613 non-null  int64   
24  has_basement        21613 non-null  int64   
25  house_age           21613 non-null  int64   
dtypes: float64(5), int64(21)
memory usage: 4.3 MB

```

Unique Values in Key Categorical Columns

```

print('Waterfront unique values:', df['waterfront'].unique())
print('View unique values:', df['view'].unique())
print('Condition unique values:', df['condition'].unique())
print('Grade unique values:', df['grade'].unique())
print('Floors unique values:', df['floors'].unique())

```

```

Waterfront unique values: [0 1]
View unique values: [0 3 4 2 1]
Condition unique values: [3 5 4 1 2]
Grade unique values: [ 7  6  8 11  9  5 10 12  4  3 13  1]
Floors unique values: [1.  2.  1.5 3.  2.5 3.5]

```

Correlation Analysis

```

## Correlation matrix
df.corr(numeric_only=True)

```

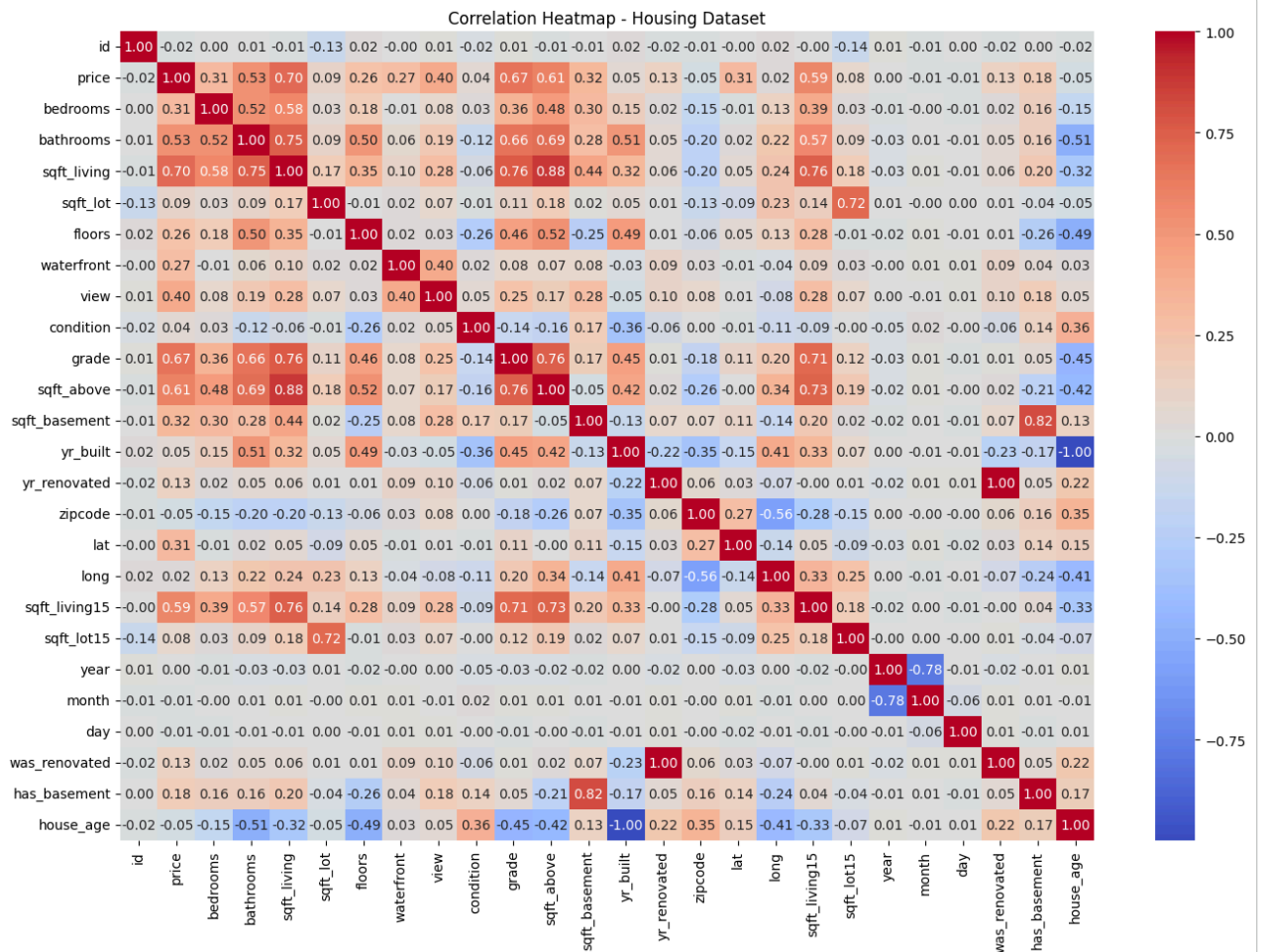
	id	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	condition	.
id	1.000000	-0.016798	0.001240	0.005158	-0.012259	-0.132109	0.018524	-0.002721	0.011592	-0.023784	
price	-0.016798	1.000000	0.308366	0.525133	0.702043	0.089655	0.256785	0.266331	0.397347	0.036391	
bedrooms	0.001240	0.308366	1.000000	0.515932	0.576693	0.031713	0.175466	-0.006578	0.079543	0.028502	
bathrooms	0.005158	0.525133	0.515932	1.000000	0.754665	0.087740	0.500653	0.063744	0.187737	-0.124982	
sqft_living	-0.012259	0.702043	0.576693	0.754665	1.000000	0.172826	0.353949	0.103818	0.284611	-0.058753	
sqft_lot	-0.132109	0.089655	0.031713	0.087740	0.172826	1.000000	-0.005201	0.021604	0.074710	-0.008958	
floors	0.018524	0.256785	0.175466	0.500653	0.353949	-0.005201	1.000000	0.023698	0.029444	-0.263768	
waterfront	-0.002721	0.266331	-0.006578	0.063744	0.103818	0.021604	0.023698	1.000000	0.401857	0.016653	
view	0.011592	0.397347	0.079543	0.187737	0.284611	0.074710	0.029444	0.401857	1.000000	0.045990	
condition	-0.023784	0.036391	0.028502	-0.124982	-0.058753	-0.008958	-0.263768	0.016653	0.045990	1.000000	
grade	0.008130	0.667463	0.356978	0.664983	0.762704	0.113621	0.458183	0.082775	0.251321	-0.144674	
sqft_above	-0.010843	0.605565	0.477614	0.685342	0.876597	0.183512	0.523885	0.072075	0.167649	-0.158214	
sqft_basement	-0.005152	0.323837	0.303112	0.283770	0.435043	0.015286	-0.245705	0.080588	0.276947	0.174105	
yr_built	0.021379	0.053981	0.154198	0.506019	0.318049	0.053080	0.489319	-0.026161	-0.053440	-0.361417	
yr_renovated	-0.016907	0.126442	0.018850	0.050739	0.055363	0.007644	0.006338	0.092885	0.103917	-0.060618	
zipcode	-0.008221	-0.053166	-0.152754	-0.203866	-0.199430	-0.129574	-0.059121	0.030285	0.084827	0.003026	
lat	-0.001891	0.306919	-0.008913	0.024573	0.052529	-0.085683	0.049614	-0.014274	0.006157	-0.014941	
long	0.020798	0.021570	0.129482	0.223042	0.240223	0.229521	0.125419	-0.041910	-0.078400	-0.106500	
sqft_living15	-0.002903	0.585373	0.391666	0.568634	0.756420	0.144608	0.279885	0.086463	0.280439	-0.092824	
sqft_lot15	-0.138798	0.082455	0.029256	0.087175	0.183286	0.718557	-0.011269	0.030703	0.072575	-0.003406	
year	0.010127	0.003553	-0.009804	-0.026596	-0.029038	0.005468	-0.022315	-0.004165	0.001364	-0.045589	
month	-0.011706	-0.010052	-0.001588	0.007392	0.011810	-0.002369	0.014005	0.008132	-0.005638	0.021978	
day	0.001939	-0.014670	-0.008015	-0.005304	-0.007246	0.000634	-0.006662	0.010518	0.011174	-0.004643	
was_renovated	-0.016944	0.126101	0.018562	0.050260	0.055094	0.007745	0.006260	0.093294	0.104062	-0.060139	
has_basement	0.002630	0.180225	0.163503	0.163534	0.204495	-0.035346	-0.256560	0.037227	0.181808	0.135577	
house_age	-0.021216	-0.053921	-0.154343	-0.506407	-0.318488	-0.052990	-0.489640	0.026093	0.053458	0.360665	

26 rows × 26 columns

```

## Heatmap of correlations
plt.figure(figsize=(14, 10))
sns.heatmap(df.corr(numeric_only=True), annot=True, fmt='.2f', cmap='coolwarm')
plt.title('Correlation Heatmap - Housing Dataset')
plt.tight_layout()
plt.show()

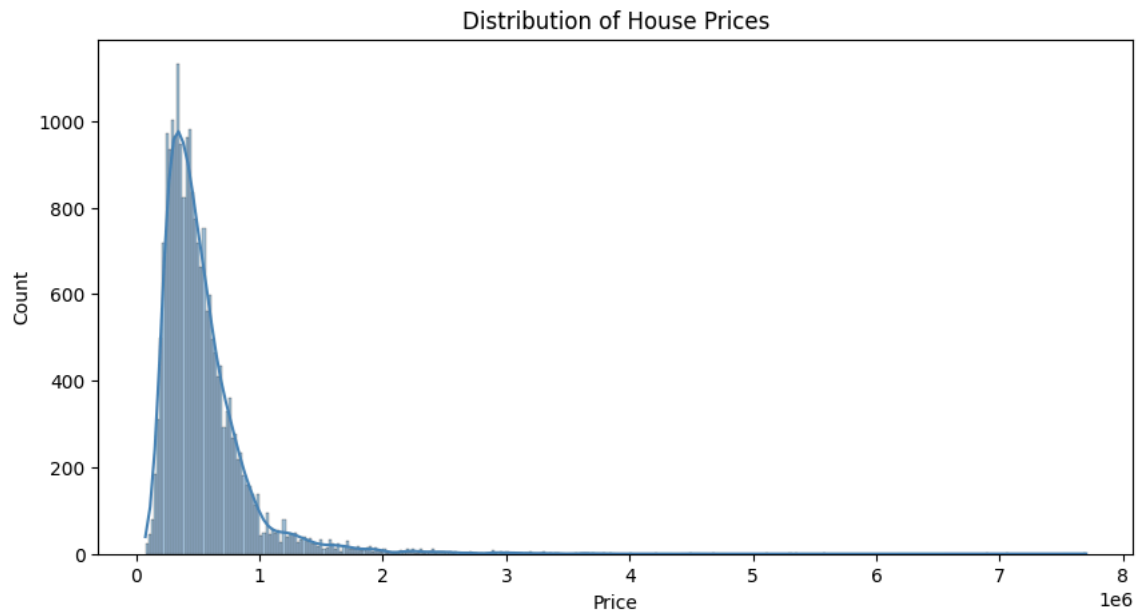
```



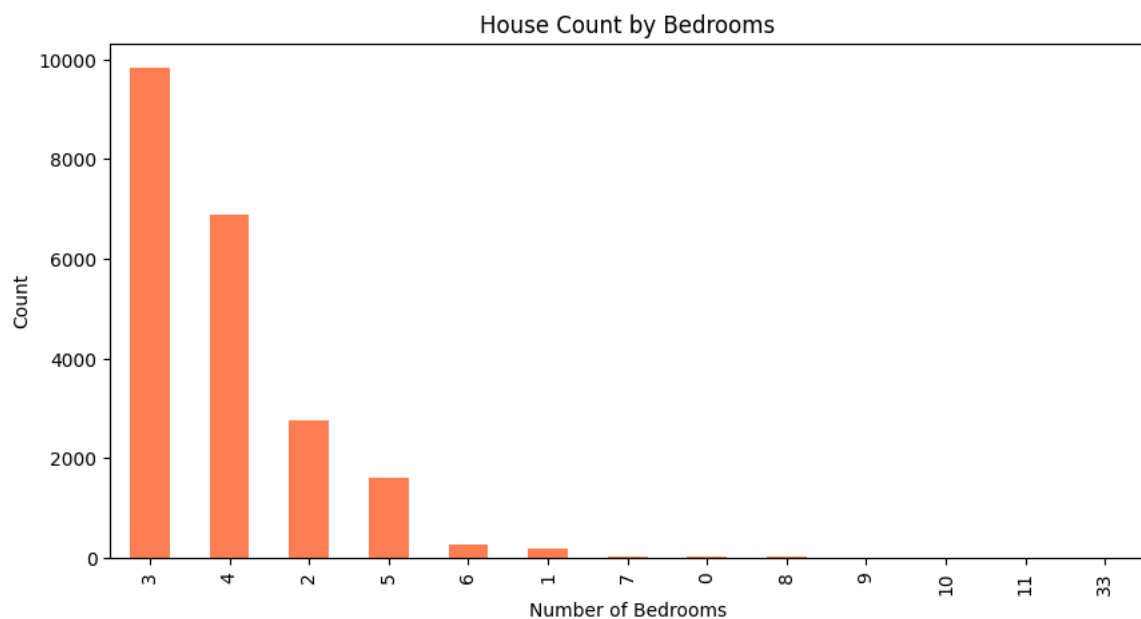
Visualization

Target Variable: Price Distribution

```
## Price distribution
plt.figure(figsize=(10, 5))
sns.histplot(df['price'], kde=True, color='steelblue')
plt.xlabel('Price')
plt.ylabel('Count')
plt.title('Distribution of House Prices')
plt.show()
```

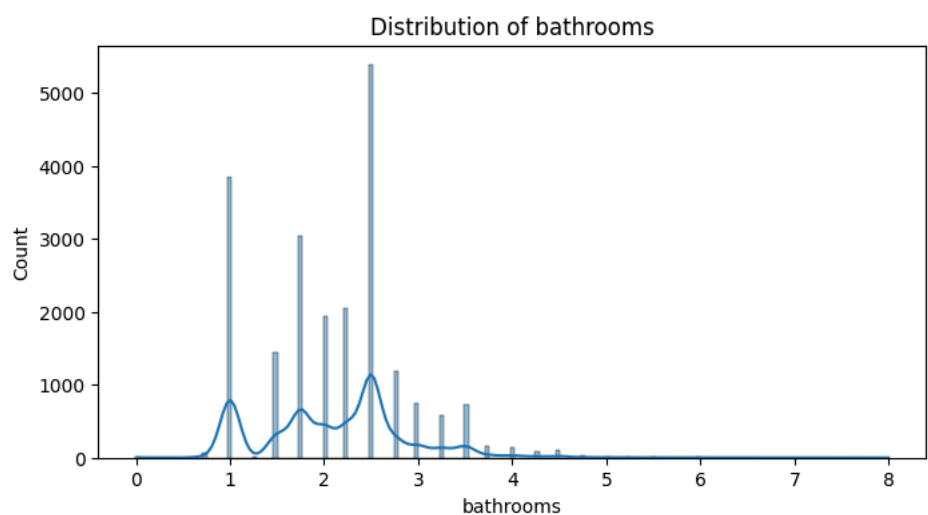
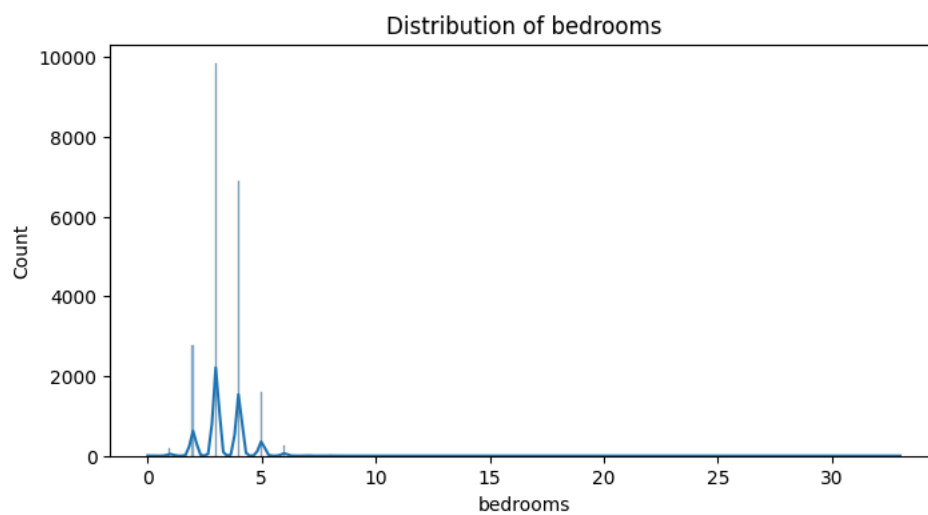
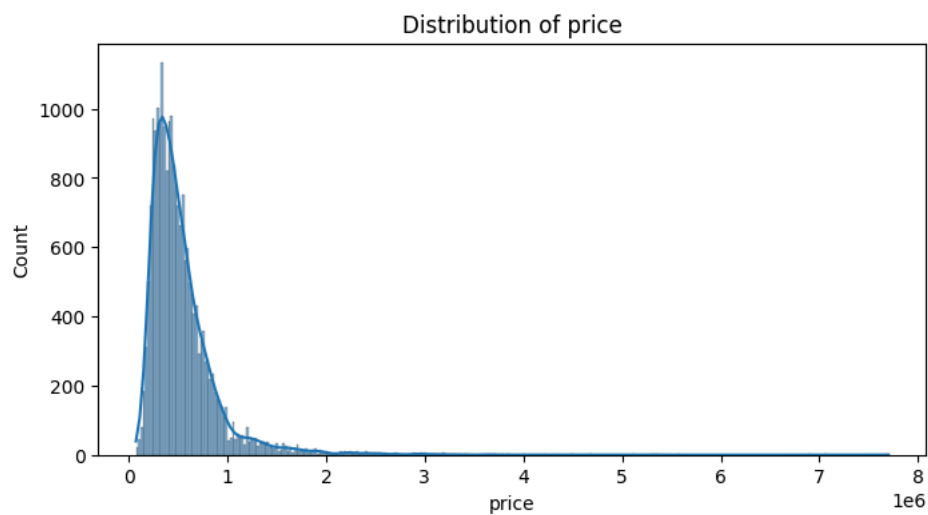
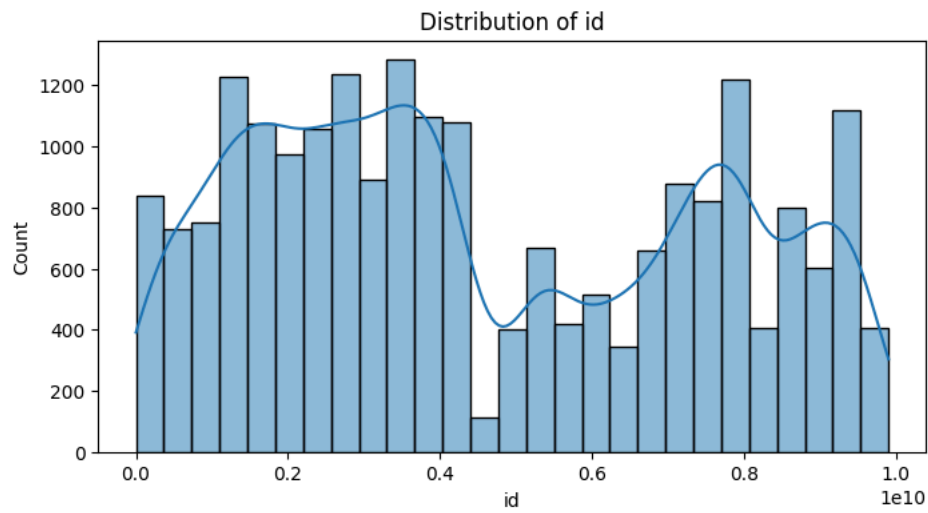


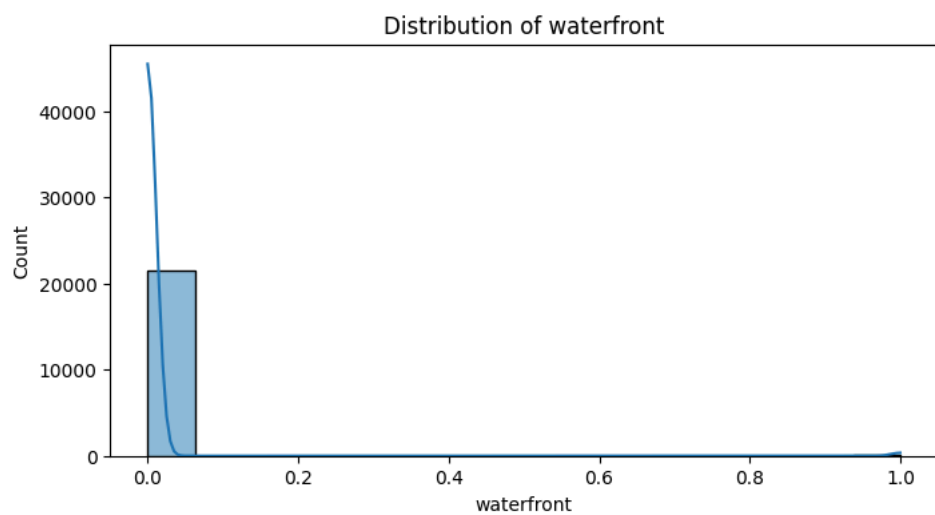
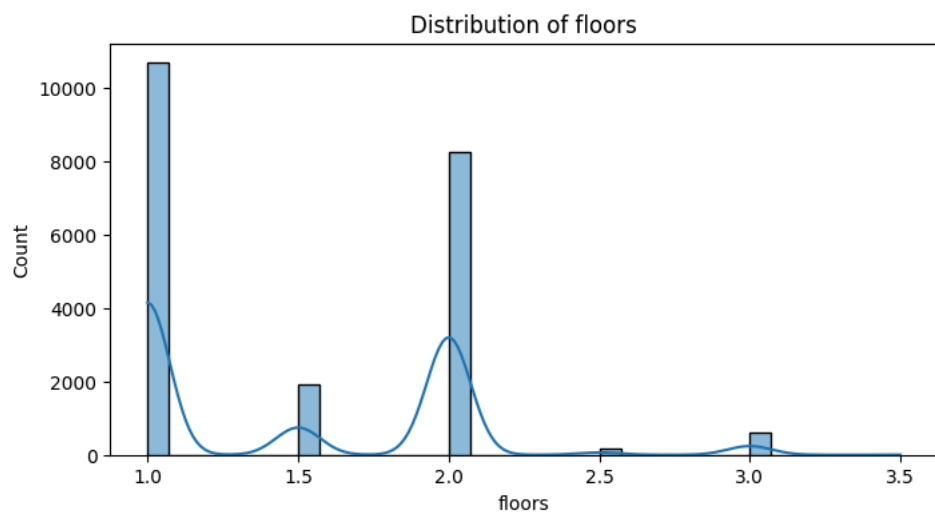
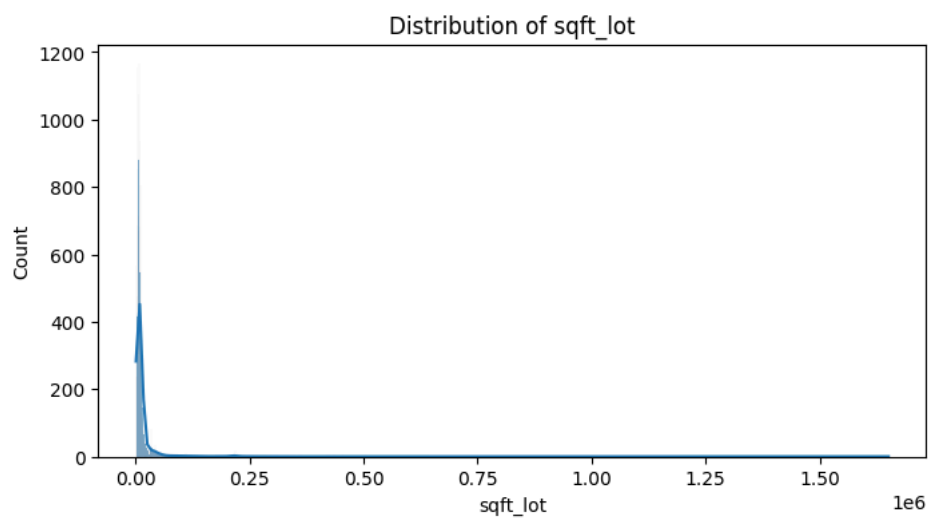
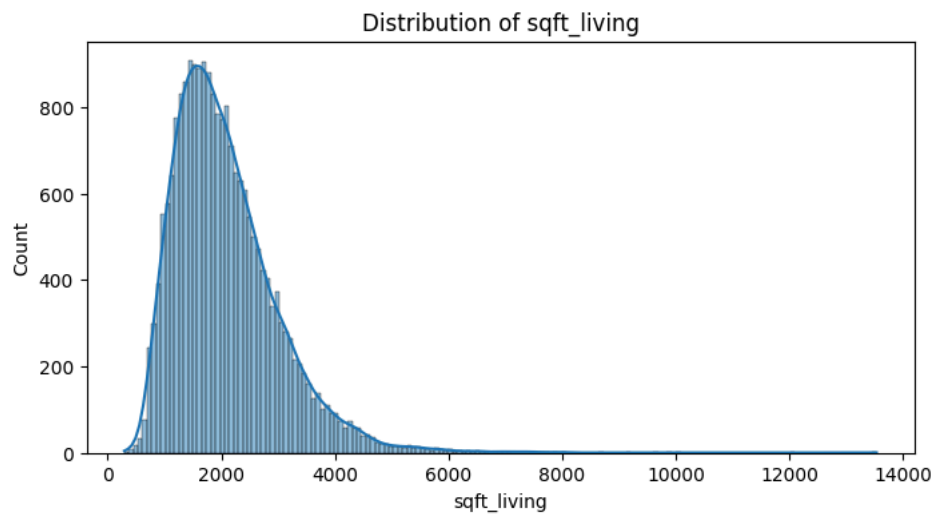
```
## Price by number of bedrooms
df['bedrooms'].value_counts().plot(kind='bar', figsize=(10,5), color='coral')
plt.xlabel('Number of Bedrooms')
plt.ylabel('Count')
plt.title('House Count by Bedrooms')
plt.show()
```

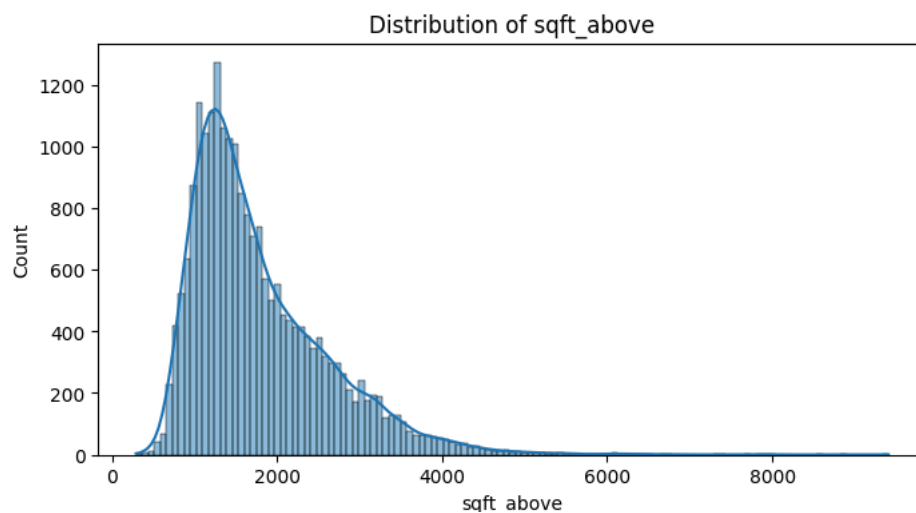
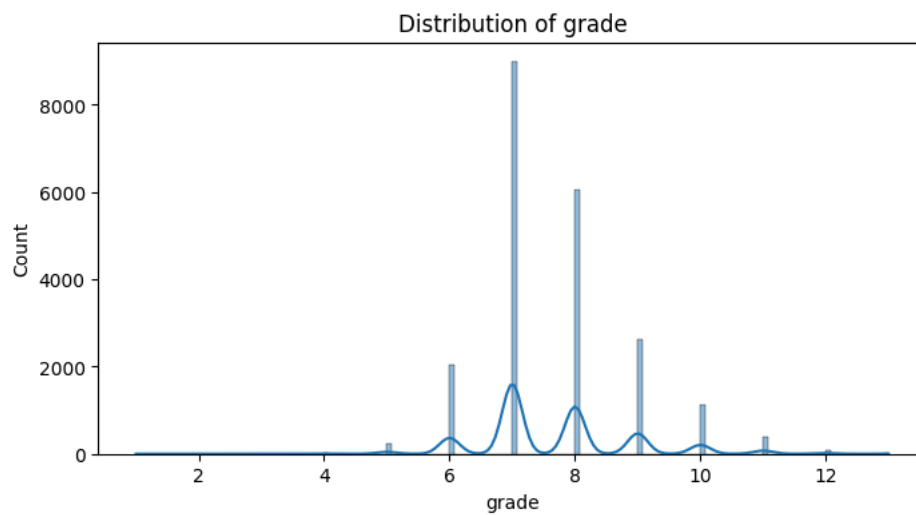
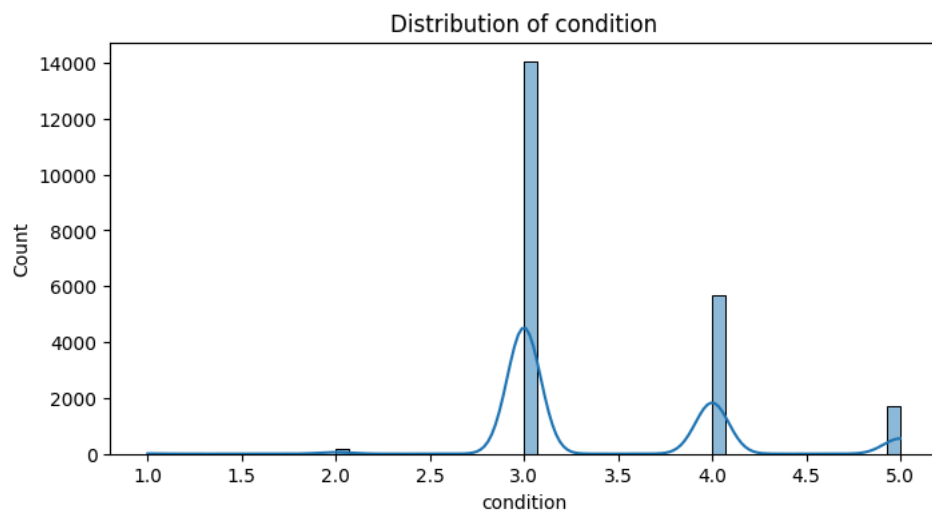
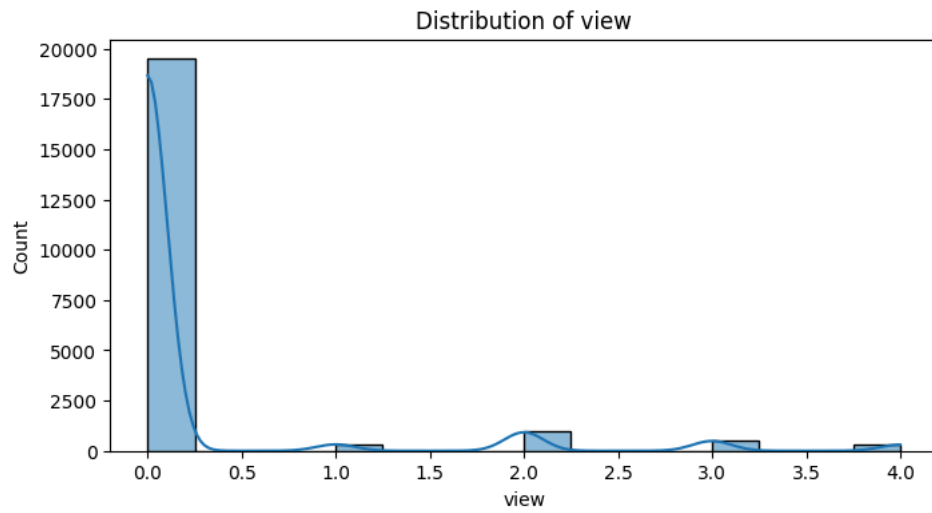


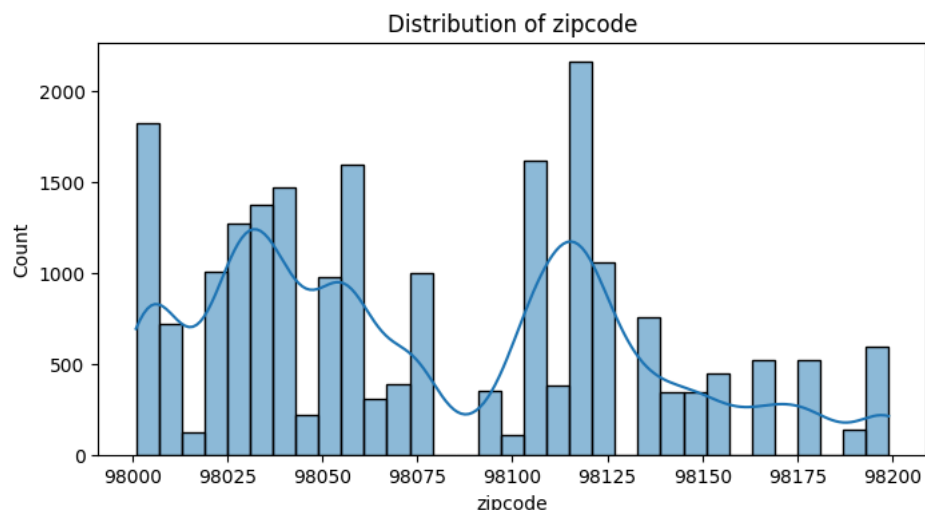
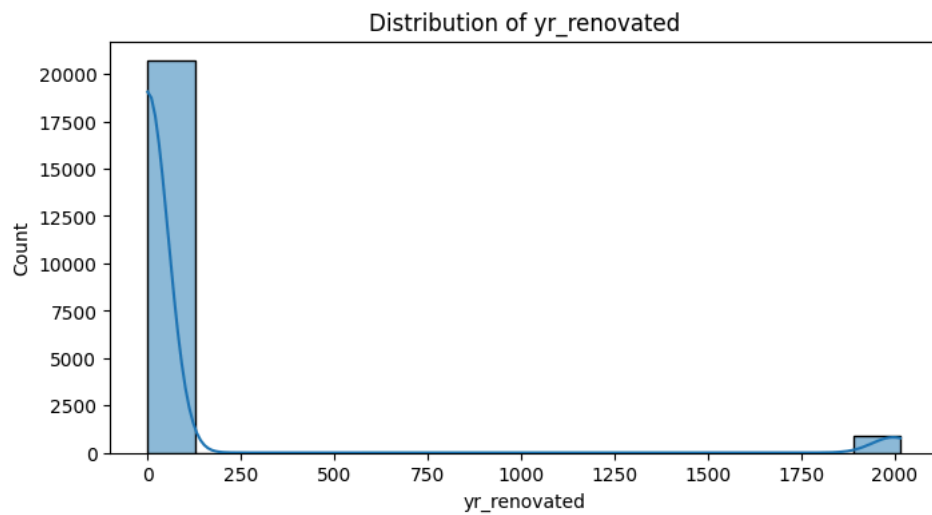
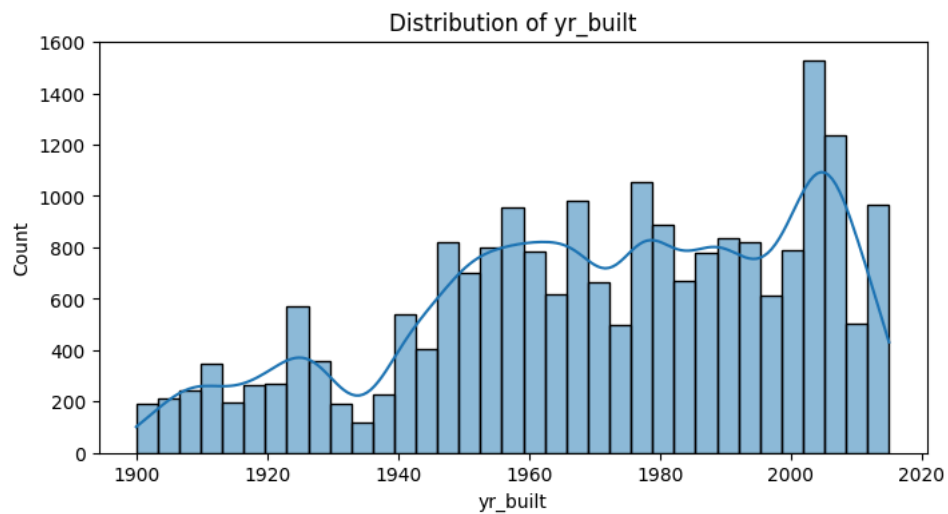
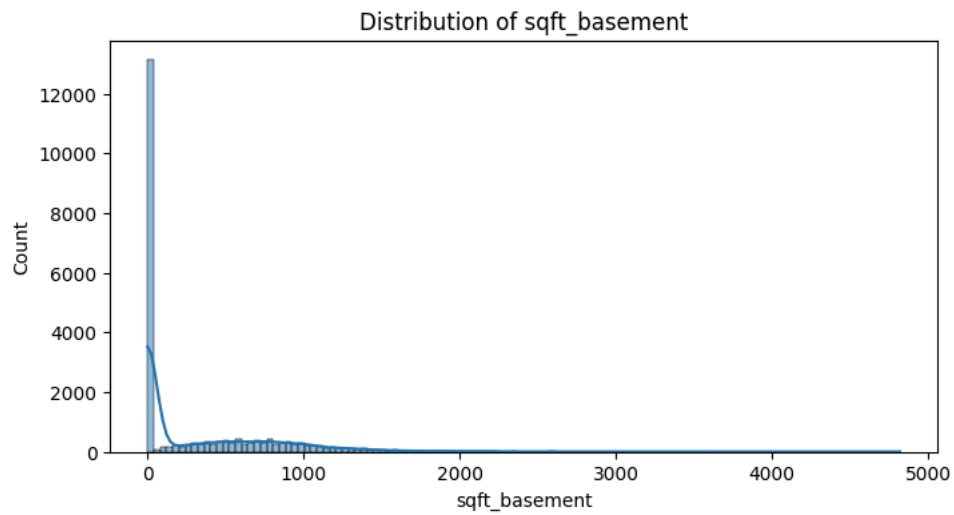
Univariate Analysis - Histograms for all columns

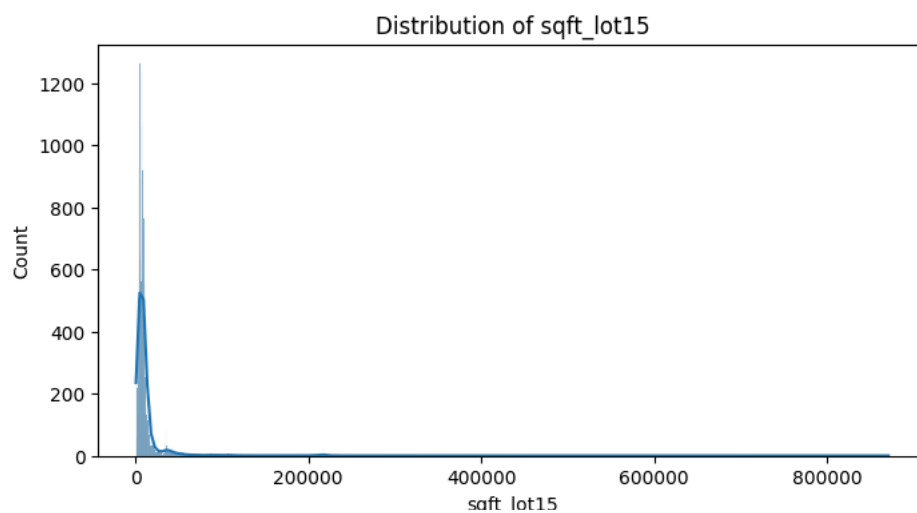
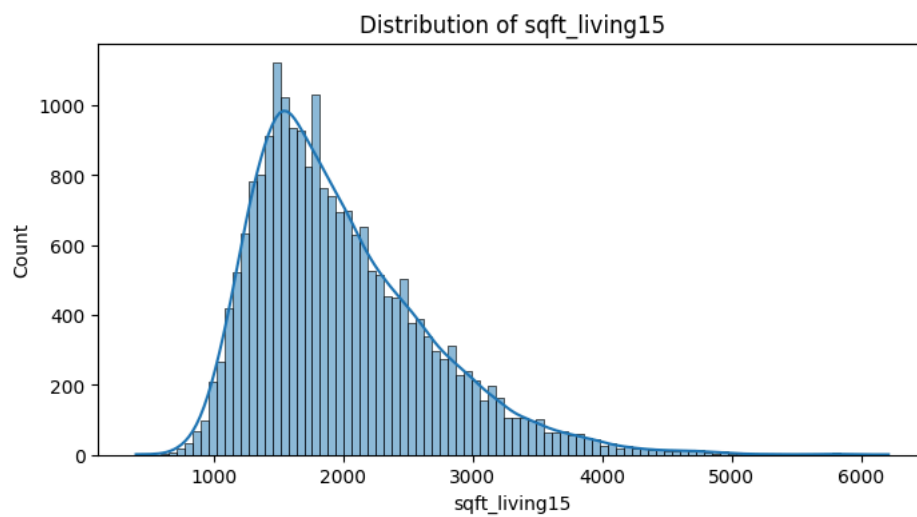
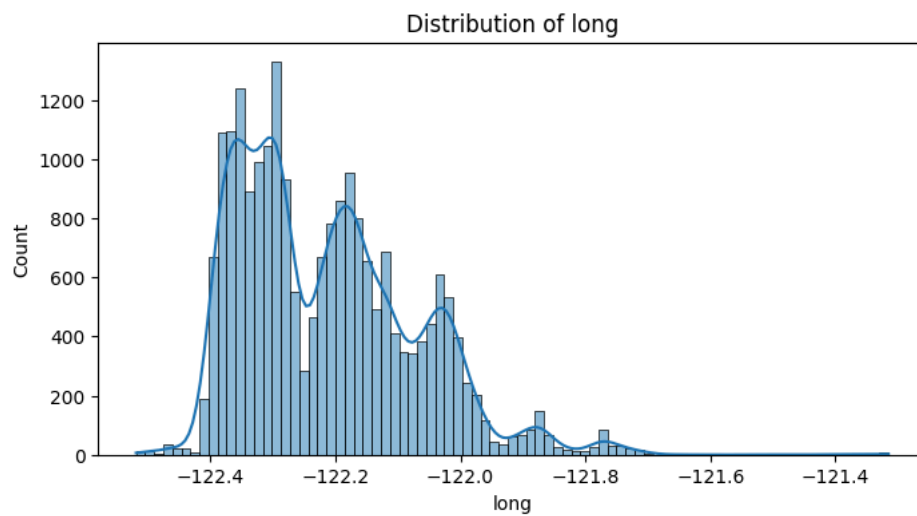
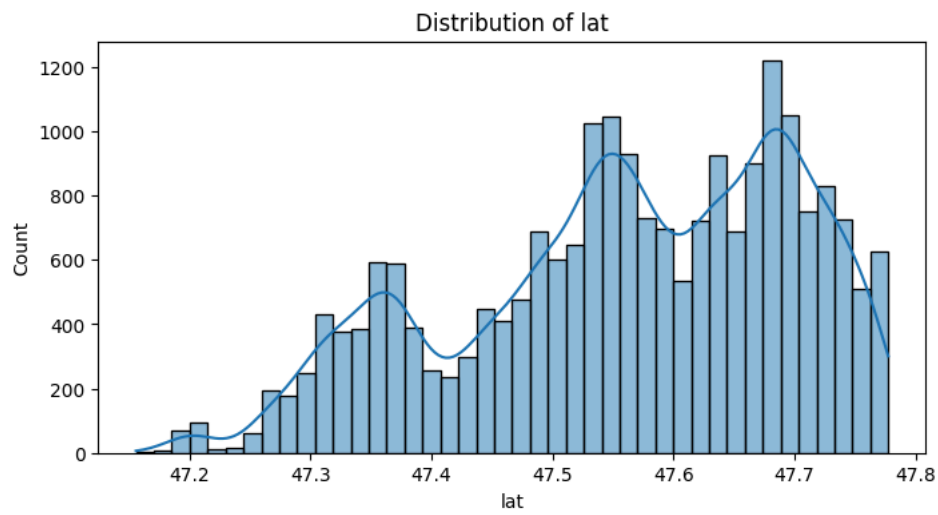
```
## Histogram for all numeric columns
for column in df.select_dtypes(include=np.number).columns:
    plt.figure(figsize=(8, 4))
    sns.histplot(df[column], kde=True)
    plt.title(f'Distribution of {column}')
    plt.show()
```

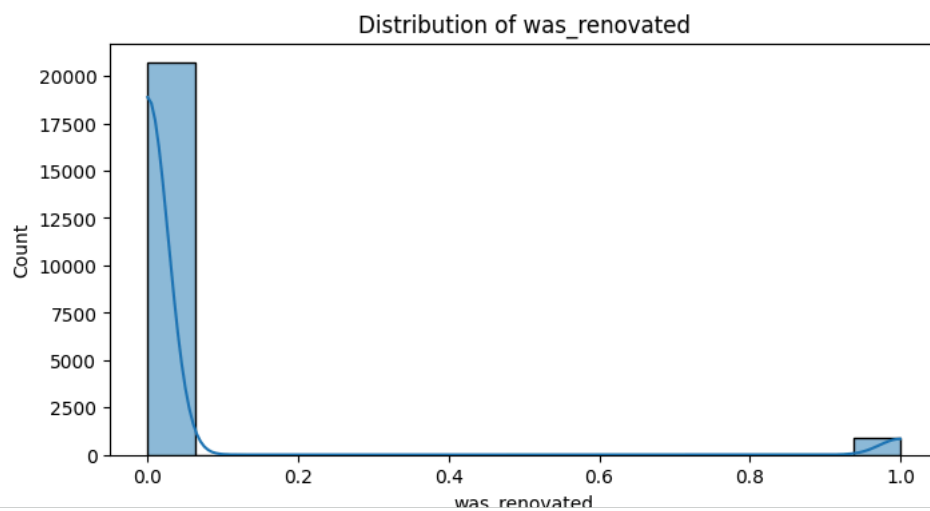
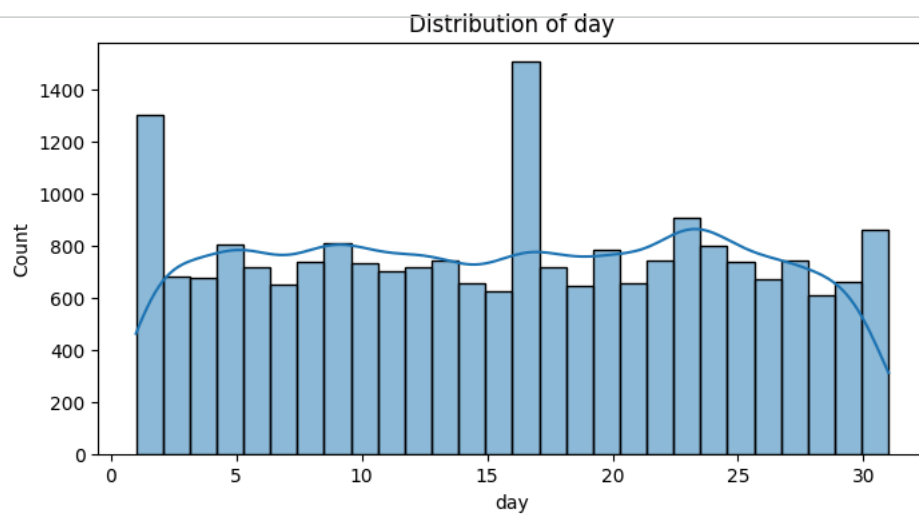
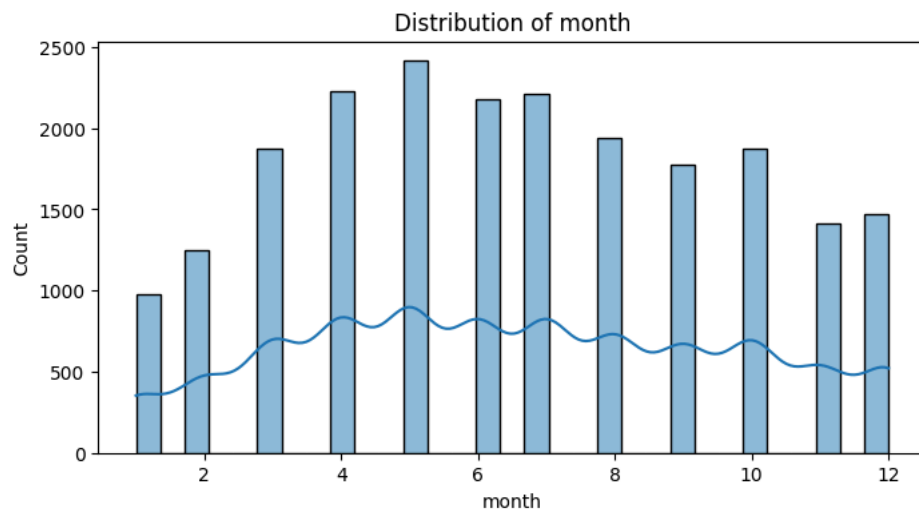
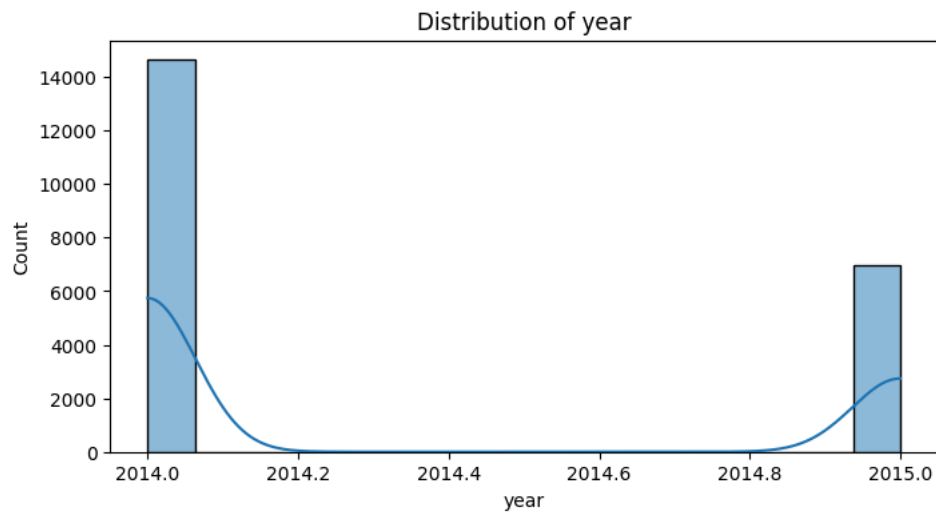







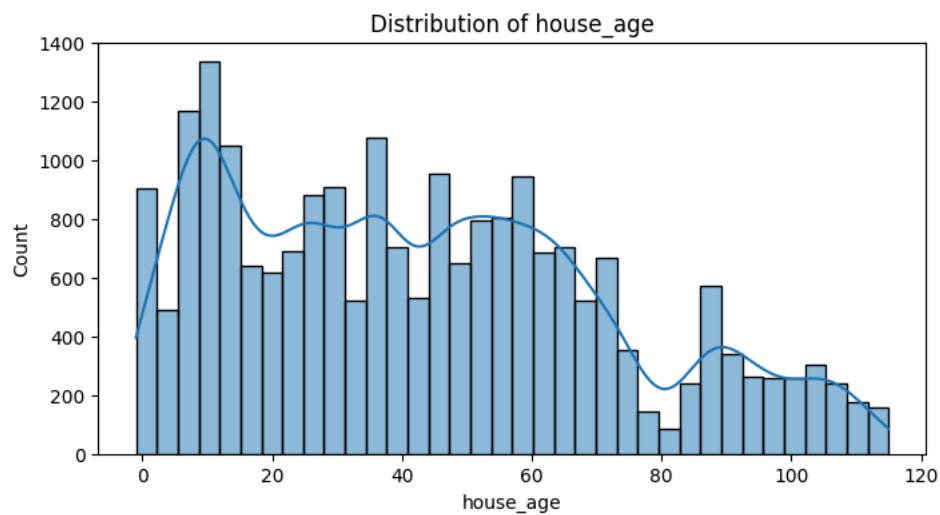
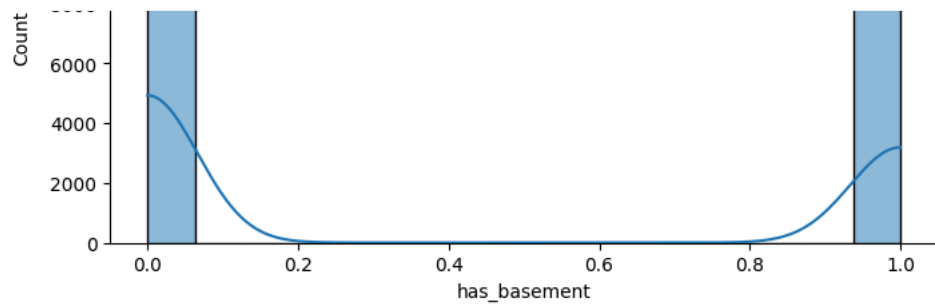


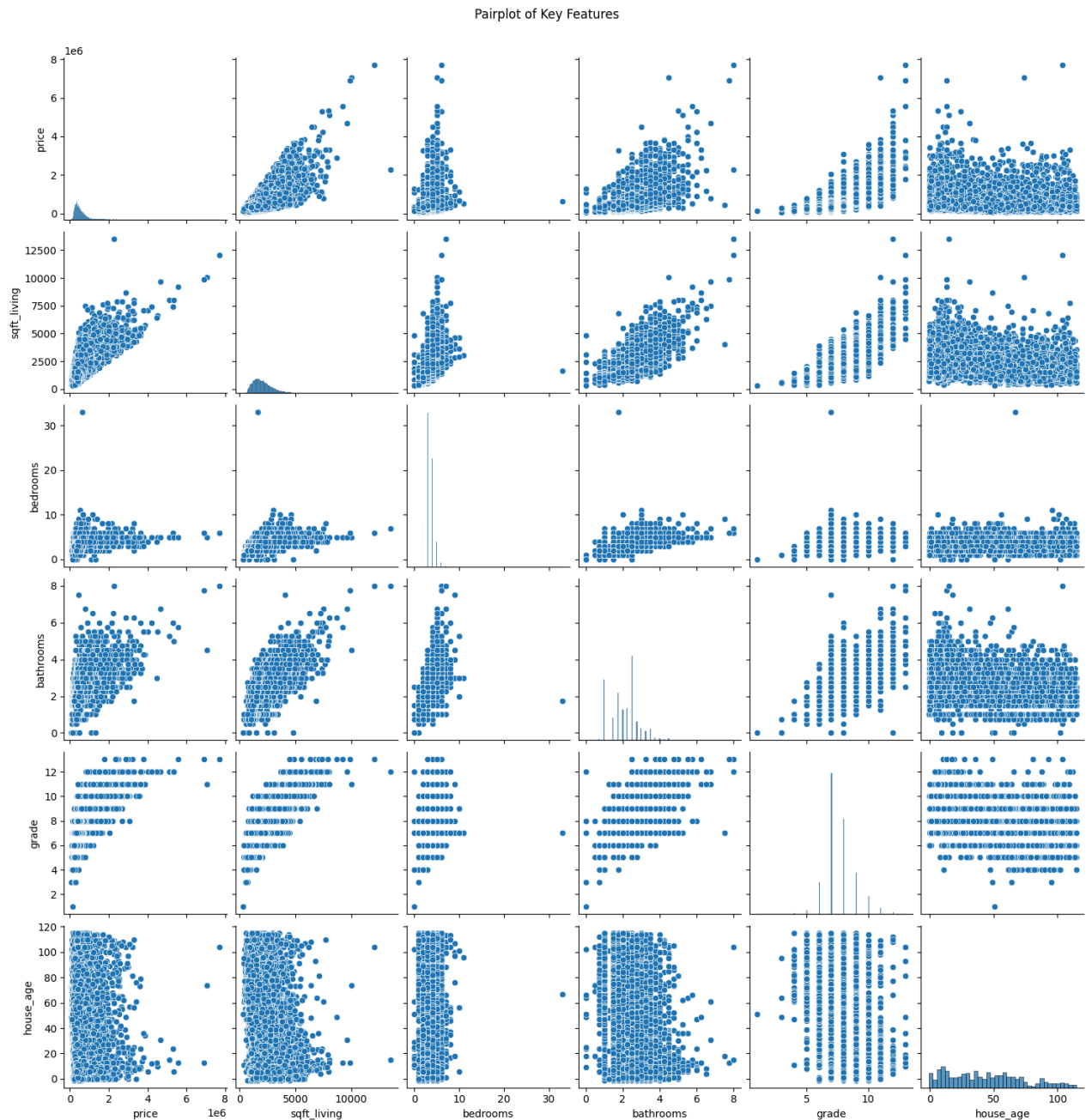




Bivariate & Multivariate Analysis Distribution of has_basement

```
## Pairplot for key features
key_features = ['price', 'sqft_living', 'bedrooms', 'bathrooms', 'grade', 'house_age']
sns.pairplot(df[key_features])
plt.suptitle('Pairplot of Key Features', y=1.02)
plt.show()
```





Category Plots

```
## Boxplot: Price vs Condition
sns.catplot(x='condition', y='price', data=df, kind='box', height=5, aspect=1.5)
plt.title('Price vs Condition')
plt.show()
```